Chapter 3: Strings and Basic I/O

So far, Vertex programs have worked with numbers and stored values under names. Now we will give those programs something to say. A string is a sequence of characters used as a text value. (1) With strings and basic input/output (I/O), a program can ask for information, use the answer, and produce a readable result.

The arithmetic and assignment machinery stays in place. We add string literals to the tokenizer and a few expression forms to the parser. The evaluator gives those forms their behavior. We also change print's spelling: this version requires `print(expression)`.

The implementation is in `chapter-03-strings-and-basic-io/code` in the working course. Its `example.v` runs without keyboard input. That gives us a repeatable example even though the language now supports interactive programs.

Breaking Changes

Programs from Programs and Assignments may need these changes:

Earlier code	What changes here
`print x`	Write `print(x)`. Parentheses are now required.
Variables named `input`, `number`, or `string`	Rename them, for example to `input_value`, `total`, or `text`. These names are now keywords for dedicated expression forms.
Ordinary variables named `__input` or `__output`	Rename them unless the special input/output behavior is intended. Input consumes and clears `__input`; print overwrites `__output`.

The special variables remain identifiers, not keywords. They now refer to global input/output state, so they are no longer ordinary storage for unrelated values. An input operation expects `__input` to contain a string.

Numeric expressions, identifier assignments, and semicolon-separated statement lists keep their earlier behavior. Line comments also carry forward: `//` outside a string starts a comment, while `"//"` is a string containing two slashes.

A Program with Text

```
__input = "Ada";
name = input("Name? ");
greeting = "Hello, " + name + "!";
print(greeting);
```

This program prints `Hello, Ada!`. The special variable `__input` supplies the answer to the input expression. Input consumes that value and clears the variable. Without a supplied value, `input("Name? ")` displays the prompt and waits for a line from the keyboard.

The assignment to `greeting` still evaluates an expression before storing its value in the environment. The expression now joins text instead of adding numbers. The syntax tree uses the same `+` nodes we already have.

Run the supplied example from its code directory:

```
./vertex example.v < /dev/null
```

Redirecting standard input from `/dev/null` makes this a useful check: an accidental keyboard read reaches end of input instead of waiting indefinitely. The example supplies a value before every input call.

String Literals and String Values

In Vertex source, a string literal uses double quotes:

```
message = "hello";
empty = "";
```

The quotes delimit the literal. They are not part of the resulting value. Likewise, an escape sequence in the source can describe a character that would be inconvenient to write directly. (2) `"line\nfeed"` contains a newline in its value, while `"say \"hello\""` contains quotation marks.

String literals must stay on one source line. Writing `\n` inside the literal is different from putting a physical line break between its quotes. Vertex decodes five escapes after tokenization: `\n` for newline, `\t` for tab, `\r` for carriage return, `\"` for a quotation mark, and `\\` for a backslash. Other escapes are syntax errors. These are Vertex's rules, not Python's string-literal rules.

Source literal	Value
`"hello"`	Five text characters
`""`	Empty text
`"a\tb"`	An `a`, a tab, and a `b`
`"a\\b"`	An `a`, a backslash, and a `b`

Single-quoted literals are not part of this Vertex grammar.

Recognizing a String Token

The new tokenizer pattern is:

```
(r'"(?:\\[^\r\n]|[^"\\\r\n])*"', "string"),
```

The outer quotes in the regular expression match the quotes in Vertex source. Inside them, the repeated group accepts either a backslash followed by a non-line-ending character, or an ordinary character that is not a quote, backslash, carriage return, or newline. That distinction lets an escaped quote belong to the string without ending it.

The tokenizer first finds the complete literal. Then this branch turns its source spelling into a value:

```
elif current_tag == "string":
    token["value"] = decode_string(value, line, column)
```

`decode_string` is our own function in `tokenizer.py`:

```
def decode_string(text, line, column):
    escapes = {"n": "\n", "t": "\t", "r": "\r", '"': '"', "\\": "\\"}
    result = ""
    position = 1
    while position < len(text) - 1:
        character = text[position]
        if character == "\\":
            position += 1
            character = text[position]
            if character not in escapes:
                raise SyntaxError(
                    f"Unknown escape \\{character} at line {line}, "
                    f"column {column + position - 1}"
                )
            character = escapes[character]
        result += character
        position += 1
    return result
```

The tokenizer has already matched both quotes, so the loop starts just after the opening quote and stops before the closing one. Ordinary characters pass through unchanged. A backslash consumes the next character too, using the dictionary to find the character that belongs in the value. An unknown escape reports the source location of its backslash.

For example, the two source characters `\n` become one newline. The source `"\\n"` instead produces a backslash followed by `n`: the decoder consumes the two backslashes together, then copies the `n`. It does not decode the result again. Python-style hexadecimal, Unicode, and octal escapes are not supported.

For `"hello"`, the token includes the tag `string`, the value `hello`, and its source line and column. An unterminated literal cannot match the string pattern and eventually produces a tokenizer error at its opening quote.

Strings in the Syntax Tree

The parser adds a string case to `parse_factor`:

```
if token["tag"] == "string":
    return {"tag": "string", "value": token["value"]}, tokens[1:]
```

The resulting abstract syntax tree (AST) represents the value without the source's quotation marks. The evaluator's matching case is just as small:

```
if ast["tag"] == "string":
    return ast["value"]
```

A literal evaluates to its own value. An identifier evaluates to the value looked up in the environment. Once either produces a string, the surrounding operation can use it.

For `"ha" * 3 + "!"`, the outer node is addition. Its left child is a multiplication of the string `ha` by the number `3`; its right child is the string `!`. Precedence has not changed. Multiplication still binds more tightly than addition, producing `hahaha!`.

Operators Depend on Their Operands

The evaluator delegates arithmetic operators to Python after evaluating both operands. That also provides the string behavior we need:

Expression	Result
`"dog" + "cat"`	`"dogcat"`
`"ha" * 3`	`"hahaha"`
`3 * "ha"`	`"hahaha"`
`"ha" * 0`	`""`
`"ha" * -2`	`""`
`"3" + "4"`	`"34"`
`3 + 4`	`7`

The parser accepts expressions such as `"ha" * 2.0`, but evaluation raises a type error. Repetition requires an integer, even when a floating-point value happens to be mathematically integral. Adding text to a number also raises a type error. Vertex does not silently convert either operand.

This separates syntax from meaning. `"hello" - 2` has the grammar of a valid expression, but the evaluator cannot subtract those values. We will use explicit conversion when we want to cross between numbers and text.

Input Produces a String

Input has two forms:

```
name = input();
name = input("Your name? ");
```

These are separate examples. Each call returns one string. Keyboard input reads one line and removes the line ending. Entering the characters `42` returns `"42"`, not the number `42`. Pressing Enter on an empty line returns `""`.

The prompt may be an expression:

```
name = input("Your " + "name? ");
```

The interpreter evaluates that expression first and requires a string result. `input(42)` therefore raises a type error. Input expressions can be nested inside other expressions or used on the right side of an assignment.

Input's parentheses resemble a function call, but this interpreter handles it as a dedicated expression. There are no general function calls or function values yet. `input`, `number`, and `string` are reserved words in this version. Names such as `input_value` and `numbered` remain ordinary identifiers.

Converting Between Text and Numbers

Explicit type conversion changes a value's representation or type at the programmer's request. (3) In Vertex, `number(s)` accepts a string and returns a numeric value. Leading and trailing whitespace are ignored. A leading sign is allowed, and a decimal point makes the result a float. Without a decimal point, the result is an integer.

Expression	Result	Type
`number("42")`	`42`	Integer
`number("-3")`	`-3`	Integer
`number("+3")`	`3`	Integer
`number(" .5 ")`	`0.5`	Float
`number("5.")`	`5.0`	Float

The plus sign in `"+3"` is part of input text. It does not add unary plus to the expression grammar. Conversion accepts decimal text, not an expression to execute. `number("2+3")`, `number("")`, and `number("hello")` raise value errors. Exponent notation such as `"1e3"` and spellings such as `"nan"` and `"inf"` are also rejected. Passing a number to `number`, as in `number(42)`, raises a type error because its argument must be a string.

The reverse operation is `string(n)`. It accepts an integer or float and returns text. For example, `string(42)` returns `"42"`, and `string(5.0)` returns `"5.0"`. Passing a string to this operation raises a type error.

```
__input = "21";
answer = number(input()) * 2;
print("Answer: " + string(answer));
```

This prints `Answer: 42`. The input expression supplies text, conversion produces an integer, multiplication computes a new number, and string conversion makes that number available for concatenation.

Print already accepts either type. `print(answer)` works without `string`. The explicit conversion is needed for the addition in `"Answer: " + string(answer)`.

The Expression Grammar

The relevant Extended Backus-Naur Form (EBNF) rules are:

```
expression ::= term { ("+" | "-") term }
term ::= unary { ("*" | "/") unary }
unary ::= "-" unary | factor
factor ::= <number> | <string> | <identifier>
       | input_expression | number_expression
       | string_expression | "(" expression ")"
input_expression ::= "input" "(" [ expression ] ")"
number_expression ::= "number" "(" expression ")"
string_expression ::= "string" "(" expression ")"
```

Braces indicate repetition. Brackets indicate an optional part. Input's argument is optional; each conversion requires exactly one argument. These rules permit `number(input())` because a conversion's argument is itself an expression. They reject `number()` and `string(1, 2)`.

The tokenizer uses `number_conversion` for the reserved word `number`, keeping it distinct from the `number` tag for numeric literals. It similarly separates `string_conversion` from a string literal's `string` tag. The parser uses those conversion tags for nodes containing an `expression` child.

For example, the tree for `number("21")` is:

```
{
    "tag": "number_conversion",
    "expression": {"tag": "string", "value": "21"},
}
```

The evaluator evaluates the child before checking its type and converting it. The rest of the precedence machinery needs no special treatment for conversions.

Printing and Visible Side Effects

The print rule is now:

```
print_statement ::= "print" "(" expression ")"
```

Parentheses are required. Earlier programs using `print x` must change to `print(x)`. Print remains a statement, so `x = print(42)` is not valid. Assignment is still a statement too. Semicolons separate statements; a newline does not replace a semicolon.

Print evaluates its expression, turns the resulting value into text, and writes that text followed by a newline. It also stores the text in `__output`:

```
if ast["tag"] == "print":
    result = evaluate(ast["expression"], environment)
    text = str(result)
    print(text)
    global_environment(environment)["__output"] = text
    return None
```

Writing to the terminal is a side effect: the program changes something we can observe outside the value of an expression. (4) The update to `__output` also makes the latest printed text available to the program and its tests. Print itself returns no Vertex expression value.

`__output` excludes the newline that print adds. It preserves newlines already inside the string. Each print replaces the previous value rather than appending to it. Save a copy before another print when you need the earlier result:

```
print(42);
saved = __output;
print("Captured: " + saved);
```

Afterward, `saved` contains `"42"` and `__output` contains `"Captured: 42"`.

Supplying Input for a Test

At program startup, the evaluator initializes `__input` and `__output` to empty strings unless the supplied global environment already defines them. Assignments to these two names update the global environment. Ordinary variables keep the assignment behavior from Programs and Assignments.

An assignment target is an identifier node. It identifies the destination rather than reading a value. The evaluator takes the name from that node, selects the global environment for `__input` and `__output`, and stores the right-hand value there. String values on the right do not turn the left side into a string target.

When `__input` is nonempty, an input expression returns its value and immediately sets it back to `""`. It does not display the prompt or read the keyboard. The prompt expression is still evaluated and must produce a string.

```
__input = "Ada";
name = input("Name? ");
print("Input after use: [" + __input + "]");
```

This prints `Input after use: []`. Another input call would read the keyboard unless a new value were assigned first. Clearing the value prevents one test's answer from accidentally satisfying a later request.

The empty string means that no supplied value is available. It cannot represent a supplied blank answer. Tests that need to simulate a blank keyboard line use Python's mock input facility instead. A non-string `__input` value raises a type error when the program requests input.

Checking the Result

Vertex has no comparison or assertion statement yet. We can arrange a test in Vertex and check its result in Python. This example uses the implementation's `evaluate_source` helper:

```
from evaluator import evaluate_source

source = '__input="21";print(number(input())*2)'
_, environment = evaluate_source(source)
assert environment["__input"] == ""
assert environment["__output"] == "42"
```

The program still prints `42`. The test checks both the result and the fact that the supplied input was consumed. Checking only output could miss a stale input value that changes the behavior of a later call.

Run the complete test suite from the code directory:

```
bash run-tests.sh
```

It checks the tokenizer, parser, and evaluator, then runs `example.v` with standard input closed and compares its full output with the expected text. The tests in `test_io_and_number.py` also check invalid conversion arguments, input clearing, and global assignments.

The command-line runner currently lets Python exceptions appear as tracebacks. Malformed source raises a syntax error; an invalid operand type raises a type error. An unsatisfied keyboard read at end of input raises `EOFError`. These are the current error-reporting behavior, rather than one-line Vertex diagnostics.

Following the Implementation

Read `tokenizer.py` first. Find the string literal pattern, `decode_string`, and the three reserved words. Then follow `parse_factor` in `parser.py`: literal nodes carry a value, conversion nodes carry an expression, and input nodes carry an optional prompt.

In `evaluator.py`, compare those cases with arithmetic. Every operation evaluates the child expressions it needs before using their values. Binary expressions evaluate their left operand before their right operand, which matters when both contain input calls.

Finally, follow `global_environment`. It walks through `$PARENT` links to find the root dictionary. The runner currently starts with one environment. The helper also makes the input and output variables global when Python tests supply a nested environment; this chapter does not yet introduce source-level scopes.

Considerations

1. What would become ambiguous if `+` silently converted numbers to strings whenever either operand was text?
2. What alternative to an empty string could indicate that no supplied input is available? How would that affect tests for blank input?

3. Can checking only `__output` prove that a program printed the right sequence of lines? What else would a test need to observe?
4. Must `number(string(n))` always recover `n`? Try a very small floating-point value and consider which numeric spellings each conversion accepts.

Practice

Answers to the practice questions follow this section.

1. Supply `"18.5"` through `__input`, add `2`, and print `"Total: 20.5"`. Capture the result through `__output` and check it from Python.
2. Read two supplied names in sequence. Set `__input` before each call and join the names with `" and "`. Check that `__input` is empty afterward.
3. Predict the results of `"2" * 3`, `number("2") * 3`, and `string(number("2") * 3)`. Give both each value and its type.
4. Explain why `"ha" * number("3")` works but `"ha" * number("3.0")` fails.
5. Add a Python test for a blank keyboard line. Explain why setting `__input` to `""` cannot supply that answer.
6. Trace `print(__output)` after another print statement. Identify when the old value is read and when the new value is stored.

The interpreter now has text values and observable interaction. Comparisons and conditional execution will let programs make decisions about those values.

Answers to the Practice Questions

1. Supply input in the environment, then check the captured output:

```
from evaluator import evaluate_source

_, environment = evaluate_source(
    'print("Total: " + string(number(input()) + 2))',
    {"__input": "18.5"})
assert environment["__output"] == "Total: 20.5"
assert environment["__input"] == ""
```

2. Set `__input` separately before each read:

```
__input = "Ada";
first = input();
__input = "Lee";
second = input();
print(first + " and " + second);
```

   Output is `Ada and Lee`; `__input` is empty afterward. Each successful supplied read consumes the supplied string once.

3. The results are `"222"` (string), `6` (number, represented by a Python integer), and `"6"` (string). Conversion changes which operation `*` performs; converting the result back to text does not repeat the input.

4. `number("3")` returns integer `3`, which Python accepts as a string repetition count. `number("3.0")` returns float `3.0`, which is rejected for repetition even though it represents a whole number. The interpreter does not implicitly convert that float to an integer.

5. Mock a blank keyboard line and check that the read actually happens:

```
from unittest.mock import patch
from evaluator import evaluate_source

with patch("builtins.input", return_value="") as read:
    _, environment = evaluate_source('line=input()', {"__input": ""})
read.assert_called_once_with()
assert environment["line"] == ""
```

   An empty `__input` means no supplied input is waiting, so it selects the keyboard path instead of supplying an empty answer.

6. In `print("first"); print(__output);`, the second print reads the old captured text, `"first"`, before storing its own output. Both terminal lines are `first`, and `__output` remains `"first"`. The capture stores the most recent print, not an accumulated transcript.

References

1. Wikipedia contributors. “String (computer science).” Wikipedia. https://en.wikipedia.org/wiki/String_(computer_science)
2. Wikipedia contributors. “Escape sequence.” Wikipedia. https://en.wikipedia.org/wiki/Escape_sequence
3. Wikipedia contributors. “Type conversion.” Wikipedia. https://en.wikipedia.org/wiki/Type_conversion
4. Wikipedia contributors. “Side effect (computer science).” Wikipedia. https://en.wikipedia.org/wiki/Side_effect_(computer_science)